Report: RT Regression Model Selection (Ridge Pooling vs Hierarchical Ridge vs Lasso Baselines)

Bioinformatics Team

January 5, 2026

1 Executive summary

We evaluate model **form** (not feature engineering) for RT regression, focusing on:

- **Accuracy**: global RMSE/MAE on realtest.
- **Uncertainty quality**: empirical coverage of nominal 95% prediction intervals and their average width.
- **Practicality**: training time and ability to support cold-start predictions.

We compare two ridge-style model forms (and legacy lasso baselines) for lib208 and lib209 under **cap100 training** and **realtest evaluation**:

- **Ridge (supercategory, sklearn)**: a per-group ridge baseline that writes coefficient summaries and an approximate predictive distribution.
- **Ridge (partial pooling, chem-linear)**: a hierarchical ridge model fit per `species` with priors pooled through `species_cluster`, and a chemistry-informed compound prior for cold-start.
- **Lasso (supercategory)** baseline from Sally's legacy model bundles (included for context).

Key result: Ridge (partial pooling, chem-linear) maintains the strong seen-compound performance of partial pooling, yields much tighter prediction intervals with coverage close to nominal (0.95), and additionally enables single-model predictions for unseen compounds via chemistry, but costs **hours of training time** (ADVI). **Ridge (supercategory, sklearn)** matches the same RMSE (ridge-equivalent point predictions) but produces **much narrower windows that under-cover** (coverage $\sim$0.92–0.94), which would systematically exclude true peaks in downstream peak assignment.

Recommendation: Use **Ridge (partial pooling, chem-linear)** as the primary model form when we can afford offline training (e.g., periodic retrains), and keep **Ridge (supercategory, sklearn)** as a fast baseline for regression checks and quick iteration.

2 Introduction

Downstream peak assignment benefits from RT predictions that are both accurate and accompanied by well-calibrated uncertainty (for candidate windowing). At the same time, we require a model form that:

- generalizes to **new species not present in training** (cold-start), and
- remains stable in the **long tail** of sparse compound/group history.

This report summarizes our current cap100 $\rightarrow$ realtest experiments, compares ridge pooling variants to legacy lasso baselines, and provides a recommendation on model form.

3 Methods

3.1 Data and group definitions

The production RT CSVs contain rows of curated RT observations with run-level covariates (`IS*`/`RS*`/`ES_` $\times$), and identifiers including:

- `species_cluster`: supercategory identifier (matrix/supercategory),
- `species`: subgroup identifier nested under `species_cluster`,
- `comp_id`: compound id.

The stage-1 artifact stores coefficient summaries per group

$$g = (\texttt{group_id}, \texttt{comp_id}),$$

where `group_id` is derived from a configured `group_col`. In this work:

- `group_col=species_cluster` (supercategory ridge),
- `group_col=species` (hierarchical ridge; nested within supercategory).

3.2 Feature set

Let $\boldsymbol{x}_i$ be the vector of run-level covariates for test row i. For pooling comparisons in this report we use **linear anchors only** (no poly2) to avoid confounding "pooling effect" with "feature expansion effect".

3.3 Models

All models are linear-in-features at prediction time and differ in grouping and pooling structure.

3.3.1 Ridge (supercategory, sklearn): per-group ridge coefficient summaries

This baseline trains an independent ridge regression for each group $g = (\texttt{species_cluster}, \texttt{comp_id})$ using the same linear run covariates. It writes a production-friendly artifact of per-group coefficient posterior summaries (`beta_hat`, `beta_cov`, `sigma2_mean`) and uses a Normal approximation to produce per-row prediction intervals. When we match the ridge penalty λ to the Bayesian ridge formulation and use the same evaluation code, the *point predictions* match up to numerical precision, but the *interval widths and coverage* can differ substantially.

3.3.2 Ridge (partial pooling): hierarchical priors for `species` within `species_cluster`

The partial pooling model keeps the subgroup grouping (`species`) but introduces hierarchical pooling for:

- **intercepts** with compound-aware structure, and
- **slope prior means** ("slope heads") with supercategory shrinkage.

Intercept prior (`intercept_prior=comp_hier_supercat`). For each subgroup c with parent supercategory $s(c)$ and each compound k:

$$b_{(c,k)} \sim \mathcal{N}\left(t_0 + \mu_c + \alpha_k, \tau_b\right),$$

with

$$\mu_c \sim \mathcal{N}\left(\mu_{s(c)}, \tau_{\mu,s(c)}\right), \quad \mu_s \sim \mathcal{N}(0, \tau_{\mu,\text{supercat}}), \quad \alpha_k \sim \mathcal{N}(0, \tau_{\text{comp}}).$$

Slope head (`slope_head_mode=cluster_supercat`). We learn a slope mean for each subgroup $\boldsymbol{m}_c \in \mathbb{R}^p$ and use it as the prior mean for group slopes $\boldsymbol{w}_{(c,k)}$. We pool slope heads through the supercategory:

$$\boldsymbol{m}_c \sim \mathcal{N}\left(\boldsymbol{m}_{s(c)}, \tau_{w,s(c)}\right), \quad \boldsymbol{m}_s \sim \mathcal{N}\left(\boldsymbol{m}_0, \tau_{w,\text{supercat}}\right).$$

Inference. We use ADVI for scalability. The exported coefficient summaries incorporate learned prior means so pooling shifts mean predictions as intended.

3.3.3 Proposed improvement: Ridge (partial pooling) with a chemistry-informed compound prior (chem-linear)

The base partial pooling model above treats the compound offsets α_k as exchangeable:

$$\alpha_k \sim \mathcal{N}(0, \tau_{\text{comp}}).$$

This is sufficient when compound k is observed in training (we can estimate α_k), but it provides no model-derived way to set α_k for a **new compound** (`comp_id` unseen at training time).

We therefore propose a chemistry-informed prior mean for α_k using ChemBERTa embeddings. This keeps the model **single-model** (no postfit regression) while enabling true compound cold-start predictions.

Indexing and identifiers. We use:

- i for observation rows in the RT CSV,
- c for subgroup (`species`) and $s(c)$ for its parent supercategory (`species_cluster`),
- k for compound (`comp_id`),
- $h(k)$ for chemical identifier (`compound`/`chem_id`) associated with compound k,
- $d \in \{1, \ldots, D\}$ for chemistry embedding coordinates ($D = 20$ for PCA-20).

In code we remap raw ids to dense indices (arrays) via sorted unique id lists; for example, a compound id k becomes an index $\kappa(k) \in \{0, \ldots, M-1\}$ into the α vector.

Chemistry embeddings. Let $\boldsymbol{e}_h \in \mathbb{R}^D$ be the ChemBERTa PCA-20 embedding vector for chemical id h. We center embeddings using the training-compound mean $\bar{\boldsymbol{e}}$:

$$\boldsymbol{z}_h = \boldsymbol{e}_h - \bar{\boldsymbol{e}}.$$

Chem-linear compound prior. We introduce a global chemistry-to-compound weight vector $\boldsymbol{\theta}_\alpha \in \mathbb{R}^D$ and define:

$$\boldsymbol{\theta}_\alpha \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \boldsymbol{I}), \quad \delta_k \sim \mathcal{N}(0, \tau_{\text{comp}}), \quad \alpha_k = \boldsymbol{z}_{h(k)}^\top \boldsymbol{\theta}_\alpha + \delta_k,$$

with centering of α and δ (mean-zero) for identifiability with t_0. Intuitively, $\boldsymbol{z}_{h(k)}^\top \boldsymbol{\theta}_\alpha$ is a chemistry-driven prior mean shared across compounds, and δ_k is a residual compound-specific deviation that is learned for *seen* compounds (so seen-compound accuracy is maintained), while contributing uncertainty for *unseen* compounds. The rest of the partial pooling model is unchanged:

$$b_{(c,k)} \sim \mathcal{N}\left(t_0 + \mu_c + \alpha_k, \tau_b\right), \quad y_{(c,k),i} = b_{(c,k)} + \boldsymbol{x}_{(c,k),i}^\top \boldsymbol{w}_{(c,k)} + \epsilon_{(c,k),i}.$$

How this enables unseen compounds. When `comp_id` is unseen at training time, the original model has no learned α_k. The chem-linear form provides a model-derived prior mean:

$$\mathbb{E}[\alpha_{\text{new}} \mid \boldsymbol{\theta}_\alpha] \approx \boldsymbol{z}_{h(\text{new})}^\top \boldsymbol{\theta}_\alpha,$$

so we can score rows for unseen compounds using only their chemical embedding. We propagate additional uncertainty using the learned τ_{comp} for unseen compounds.

3.3.4 Lasso baselines

We include one legacy eslasso baseline for context:

- **Lasso (supercategory)**: one lasso per `species_cluster`.

Lasso "intervals" are window-based (`pred` $\pm$ `window`) rather than derived from a probabilistic predictive distribution. We still report their empirical "coverage95" for comparison, but it should not be interpreted as nominal calibration in the same way as ridge models.

3.4 Evaluation metrics

Let y_i be observed RT and $\hat{y}_i$ be the point prediction on realtest (units: minutes).

- **RMSE**: $\sqrt{\frac{1}{n}\sum_i(\hat{y}_i - y_i)^2}$.
- **MAE**: $\frac{1}{n}\sum_i |\hat{y}_i - y_i|$.
- **Coverage@95%** (ridge models): with predicted standard deviation $\hat{\sigma}_i = \sqrt{\sigma_g^2 + \boldsymbol{x}_i^\top \widehat{\mathrm{Cov}}(\boldsymbol{\beta}_g)\boldsymbol{x}_i}$, we define the nominal 95% interval as $\hat{y}_i \pm 1.96\,\hat{\sigma}_i$ and compute the fraction of test points inside.
- **Mean interval width** (ridge models): $\frac{1}{n}\sum_i 2 \cdot 1.96 \cdot \hat{\sigma}_i$.
- **Coverage@95% / width (lasso)**: computed using the legacy window: $|\hat{y}_i - y_i| \le w_i$ with width $2w_i$.

We stratify by **training support bin** of each (`group_id`, `comp_id`) group using bins:

$$\{\le 1,\ 2,\ 3\text{–}5,\ 6\text{–}10,\ 11\text{–}20,\ 21\text{–}50,\ 51\text{–}100,\ > 100\}.$$

3.5 Reproducibility

Ridge experiments were generated by:

```
./scripts/run_rt_prod.sh
```

This trains the chem-linear partial pooling model, evaluates on realtest, regenerates plots, syncs the report images under `docs/models/images/rt_pymc_multilevel_pooling_report/`, and rebuilds this PDF. To skip rebuilding the PDF, pass `--no-build-pdf`. To train only, pass `--no-eval`. The lasso baseline is evaluated via external eslasso weights under `external_repos/sally/new_models/eslasso`. and plots/tables in this report were regenerated by:

```
./scripts/plot_rt_multilevel.sh --cap 100 --libs 208,209
```

using the run directory pointer in `output/rt_prod_latest.txt`.

To run the full end-to-end report experiment suite (prod runs + subset holdouts + zero-shot) and optionally rebuild this PDF:

```
poetry run python -u scripts/run_rt_report_all.py --skip-existing --build-pdf
```

or equivalently:

```
./scripts/run_rt_report_all.sh
```

4 Results

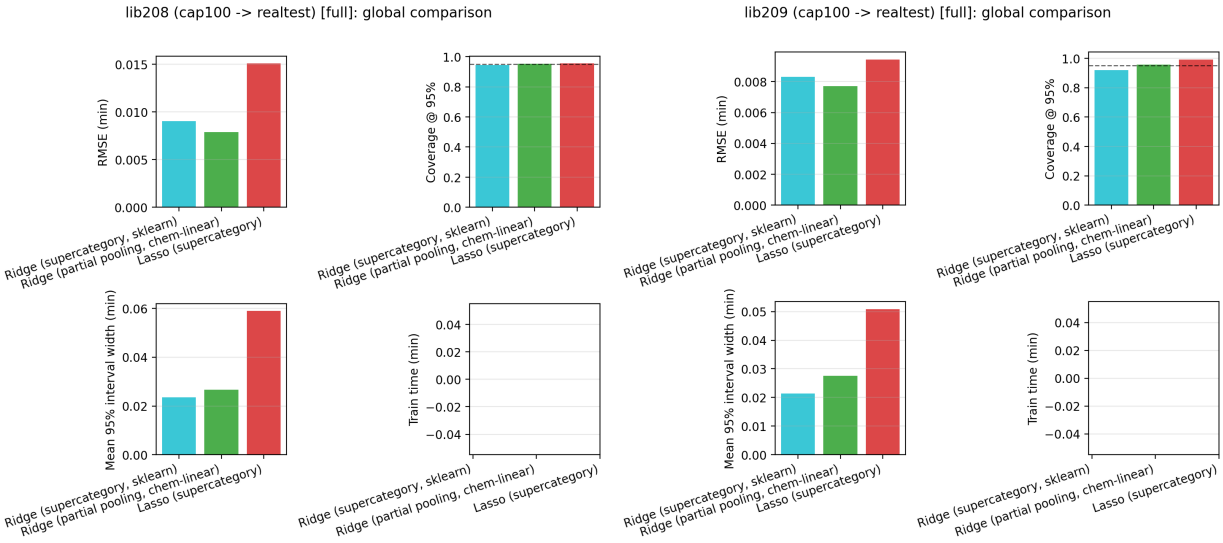
4.1 Global performance

Table 1 summarizes global metrics on realtest for lib208 and lib209. Lasso baselines do not score all rows; **Rows scored** reflects applicability.

Table 1: Global realtest metrics for cap100-trained models (linear features).

Lib	Model	RMSE	MAE	Cov95	Width95	Rows scored	Train (min)
208	Ridge (supercategory, sklearn)	0.009023	0.004673	0.943	0.023534	100.0%	0.0
208	Ridge (partial pooling, chem-linear)	0.007874	0.003821	0.953	0.026681	100.0%	138.6
208	Lasso (supercategory)	0.015081	0.007956	0.954	0.058908	94.7%	–
209	Ridge (supercategory, sklearn)	0.008317	0.004677	0.920	0.021393	100.0%	0.0
209	Ridge (partial pooling, chem-linear)	0.007707	0.004285	0.956	0.027518	100.0%	242.5
209	Lasso (supercategory)	0.009439	0.004954	0.993	0.050886	97.8%	–

Interpretation. Relative to Ridge (supercategory, sklearn), Ridge (partial pooling, chem-linear) reduces RMSE by 0.00115 min (0.069 s) on lib208 and 0.00061 min (0.037 s) on lib209, while moving coverage closer to the nominal 0.95 target. This is desirable when the prediction interval is used as an RT filter in peak assignment. Lasso (supercategory) is included as a legacy baseline: it does not score all rows (no matching model for some compounds) and its uncertainty is a fixed window rather than a probabilistic interval.



(a) lib208 (b) lib209

Figure 1: Global comparison across report baselines (cap100 training, realtest evaluation): Ridge (supercategory, sklearn), Ridge (partial pooling, chem-linear), and Lasso (supercategory). Panels show RMSE, coverage@95%, mean interval width, and training time.

4.2 Interval calibration: sklearn ridge vs partial pooling

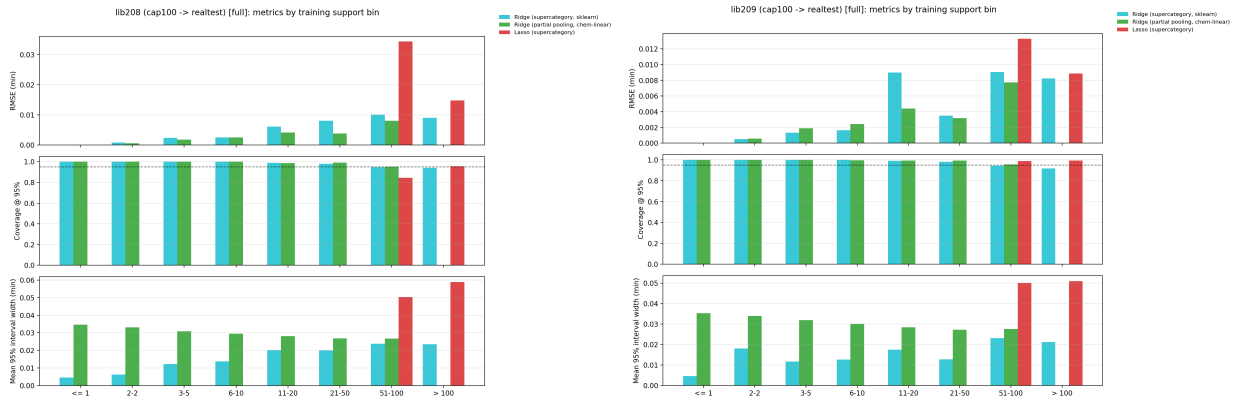
The sklearn baseline and the PyMC partial pooling model are both linear ridge-style models over the same run covariates, but they differ in how they share information across sparse groups and how they form prediction intervals.

However, the **prediction intervals** differ because they depend on how we estimate (and propagate) predictive variance. In particular, the sklearn coefficient-summary artifact estimates a separate noise scale (`sigma2_mean`) for each (`species_cluster`, `comp_id`) group, which can lead to **overly optimistic** intervals in practice. Empirically this shows up as substantially **narrower windows** but **lower-than-nominal** coverage (0.92–0.94). In a peak-assignment setting where we use the interval as a hard RT gate, this under-coverage translates directly into a higher *miss rate*: the true peak is excluded from consideration more often.

The same pattern holds across the long tail in Figure 2: the sklearn baseline tends to be narrower but under-calibrated, while PyMC partial pooling maintains near-nominal coverage with modestly wider intervals.

4.3 Performance by training support bin

Figure 2 breaks metrics down by training support bin (long tail to head).

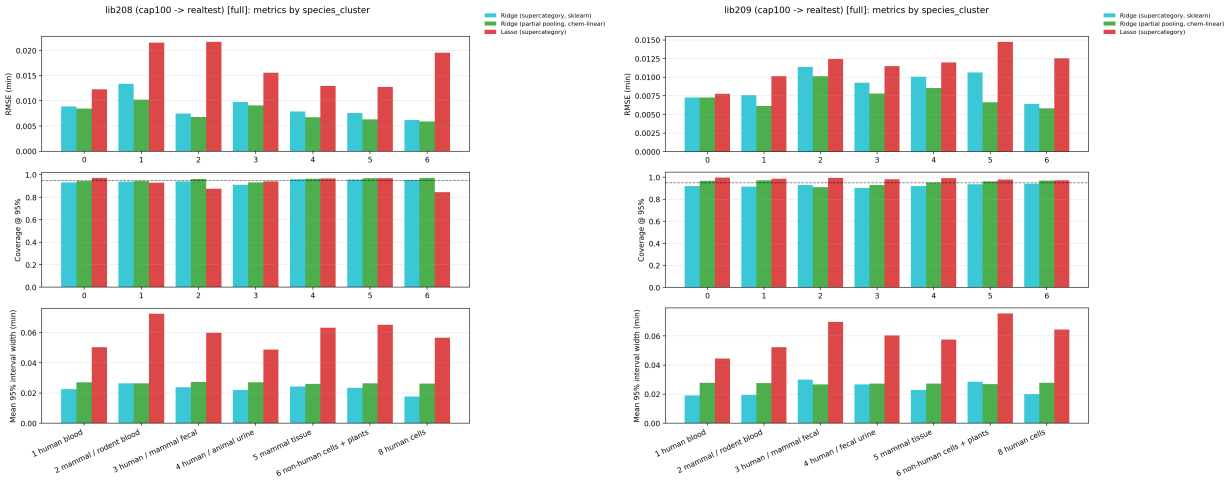


(a) lib208 (b) lib209

Figure 2: Metrics by training support bin for report baselines. Coverage@95% is empirical coverage of the nominal 95% prediction interval (ridge models) or window interval (lasso baseline).

4.4 Performance by species_cluster

Figure 3 aggregates metrics by `species_cluster`.



(a) lib208 (b) lib209

Figure 3: Metrics by `species_cluster` (supercategory) for report baselines. These aggregates reflect within-supercategory performance stability and highlight where models under/over-estimate uncertainty.

4.5 Unseen species holdout experiments (production subset)

The cap100 → realtest results above assess generalization across runs, but do not directly answer the "cold-start" question for **new species**: how do models behave when test rows belong to species unseen at training time?

We therefore run additional **group holdout** experiments on a deterministic subset of the cap100 production CSV (clusters 4 and 5, top-5 species per cluster, top-100 `comp_id`; seed=42; holdout_frac=0.2). We evaluate the following holdout modes:

- `species`: hold out entire species within each `species_cluster`.
- `species_comp`: hold out `(species, comp_id)` pairs but keep `(species_cluster, comp_id)` seen in training (new compound for a particular species; supercategory has data).

Subset construction (deterministic). We first select a small production subset to make group holdouts and evaluation fast and repeatable:

- Restrict to `species_cluster` in $\{4, 5\}$.
- Within each selected `species_cluster`, keep the top $N = 5$ `species` by row count.
- Globally, keep the top $M = 100$ `comp_id` by row count.

This yields ∼95k rows for lib208 and ∼98k rows for lib209 in the runs used for this report.

Split definitions (exact setup). All splits are created on the subset CSV with `seed=42` and `holdout_frac=0.2` (see `scripts/experiments/rt/run_unseen_generalization_prod_subset.py`):

- `species`: for each `species_cluster` s, sample $\max(1, \lfloor 0.2 \cdot |C_s| \rfloor)$ species ids to hold out (leaving at least one species in training per cluster), where C_s is the set of species observed in cluster s. Test contains all rows with held-out species.
- `species_comp`: sample eligible `(species_cluster, comp_id)` pairs that have ≥ 2 distinct species observed; for each chosen `(cluster, comp_id)` pair, sample exactly one species and hold out that `(species, comp_id)` pair. This guarantees the corresponding `(species_cluster, comp_id)` group remains present in training.

We compare:

- **Ridge (partial pooling, chem-linear)** evaluated with an explicit **unseen-species backoff**: when a held-out species makes `(species, comp_id)` unseen, we score using a derived `(species_cluster, comp_id)` coefficient obtained by aggregating fitted `(species, comp_id)` coefficients across training species.
- **Ridge (supercategory, sklearn)** evaluated with a **comp-id backoff** for unseen `(species_cluster, comp_id)` groups.
- **Lasso (supercategory)** from the legacy external bundle (not retrained on these splits; included for context).

We do **not** include the **Lasso (species)** baseline here because it is not applicable to these splits and does not score all rows (it depends on a separate local-model mapping and requires a fallback model anyway).

Tables 2 and 3 summarize these results. Compound cold-start (unseen chemistry) is handled separately in Section 4.6.

Reproducibility (commands). From the repository root, the tables in this section are generated by running:

```
# lib208 (chem-linear; default partial pooling model going forward)
poetry run python -u scripts/experiments/rt/run_unseen_generalization_prod_subset.p
  --data-csv repo_export/lib208/cap100/merged_training_all_lib208_cap100_chemclass_
  --lib-id 208 --output-dir output/rt_unseen_generalization_chemlinear_lib208_advi5
  --modes species,species_comp --train-method advi --advi-steps 5000 \
  --seed 42 --holdout-frac 0.2 --clusters 4,5 --species-per-cluster 5 --top-comp-id
  --chem-embeddings-path resources/metabolites/embeddings_chemberta_pca20.parquet \
  --theta-alpha-prior-sigma 1.0 --skip-existing
```

and similarly for lib209 by swapping `lib208` → `lib209`. Each run writes:

- the subset CSV under `<output-dir>/subset/`,
- train/test splits under `<output-dir>/splits/<mode>/`,

- trained artifacts under `<output-dir>/models/<mode>/`,
- evaluation JSONs under `<output-dir>/eval/<mode>/`,
- and a combined summary at `<output-dir>/summary.json`.

Interpretation. When the test rows belong to an *unseen species* but the `(species_cluster, comp_id)` pair is seen in training (modes `species` and `species_comp`), partial pooling can back off to a *compound-specific* supercategory coefficient (derived by aggregating fitted `(species, comp_id)` coefficients across training species). This makes partial pooling behave similarly to the supercategory ridge baseline in these modes. Unseen-compound (cold-start chemistry) behaviour is evaluated separately in Section 4.6.

Table 2: Unseen holdout metrics (lib208 production subset; seed=42; holdout_frac=0.2; clusters=4,5; top_comp_ids=100).

Holdout	Model	RMSE	MAE	Cov95	Rows scored (%)
species	Ridge (partial pooling, chem-linear)	0.014968	0.009012	1.000	100.0
species	Ridge (supercategory, sklearn)	0.011090	0.005202	0.877	100.0
species	Lasso (supercategory, external)	0.007913	0.004174	0.995	94.8
species_comp	Ridge (partial pooling, chem-linear)	0.013474	0.009569	1.000	100.0
species_comp	Ridge (supercategory, sklearn)	0.013272	0.006532	0.812	100.0
species_comp	Lasso (supercategory, external)	0.007671	0.004331	0.999	94.9

Table 3: Unseen holdout metrics (lib209 production subset; seed=42; holdout_frac=0.2; clusters=4,5; top_comp_ids=100).

Holdout	Model	RMSE	MAE	Cov95	Rows scored (%)
species	Ridge (partial pooling, chem-linear)	0.008381	0.005281	1.000	100.0
species	Ridge (supercategory, sklearn)	0.008268	0.004870	0.902	100.0
species	Lasso (supercategory, external)	0.006779	0.004267	0.996	92.4
species_comp	Ridge (partial pooling, chem-linear)	0.011684	0.006456	1.000	100.0
species_comp	Ridge (supercategory, sklearn)	0.013582	0.006320	0.883	100.0
species_comp	Lasso (supercategory, external)	0.009482	0.005677	0.997	95.0

4.6 Zero-shot chemistry-feature baselines (single model)

Section 4.5 focuses on **unseen species** generalization. Here we isolate the **true chemical cold-start** scenario where a chemical identifier (`compound`/`chem_id`) is never seen in training. We hold out 20 `chem_id` values entirely from training (selected by stratifying held-out chemicals across the RT range; seed=42) and train **single models** that can still score those held-out chemicals because chemistry enters as features (ChemBERTa PCA-20 embeddings), not via per-compound lookup tables.

We compare only:

- **Ridge (partial pooling, chem-linear; PyMC)**: our single-model RT ridge with a chemistry-informed compound prior for unseen compounds (and calibrated posterior uncertainty for prediction intervals),
- **MLP (ChemBERTa PCA-20 + cluster interactions)**: a strong non-linear single-model baseline (point prediction only).

How to improve unseen compounds without sacrificing seen compounds (single PyMC model). The seen-vs-unseen goals are not contradictory, but the chemistry head must be stabilized so it does not over-extrapolate on held-out compounds. The most direct single-model improvements to try next are:

- **Add a class anchor inside the model**: $\alpha_k = u_{\texttt{class}(k)} + \boldsymbol{z}_{h(k)}^{\top}\boldsymbol{\theta}_\alpha + \delta_k$ with hierarchical priors on $u_{\texttt{class}}$, so unseen compounds can fall back to a class mean when embeddings are weak.
- **Stronger shrinkage for $\boldsymbol{\theta}_\alpha$**: reduce `theta_alpha_prior_sigma` (or use a global shrinkage prior) to prevent high-variance zero-shot predictions while keeping δ_k so seen-compound accuracy is maintained.
- **Revisit the embedding representation**: try more PCA dimensions and/or remove whitening to reduce brittle OOD directions; PCA-20 + whitening is a pragmatic compression but may amplify tail components.
- **Allow mild non-linearity**: replace the linear head with a tiny MLP inside PyMC (still single-model) if a linear chemistry-to-compound mapping is insufficient.

Table 4: Zero-shot chemistry results on the lib208 production subset (hold out 20 `chem_id`; seed=42).

Model	RMSE	MAE	Cov95
Ridge (partial pooling, chem-linear; PyMC)	1.049358	0.699639	1.000
MLP (ChemBERTa PCA-20 + cluster interactions)	0.468125	0.345321	–

Table 5: Zero-shot chemistry results on the lib209 production subset (hold out 20 `chem_id`; seed=42).

Model	RMSE	MAE	Cov95
Ridge (partial pooling, chem-linear; PyMC)	1.523705	1.062139	0.991
MLP (ChemBERTa PCA-20 + cluster interactions)	1.061293	0.799969	–

Interpretation. This experiment directly tests whether chemistry features can support single-model scoring for unseen compounds. In these subset runs, the MLP materially outperforms the current PyMC chem-linear model on point metrics, suggesting that the chemistry head inside the PyMC hierarchy still needs improvement to match the best embedding-only predictors (while the PyMC model retains the advantage of calibrated posterior uncertainty for downstream windowing).

5 Discussion

Accuracy vs uncertainty for peak assignment. For downstream peak assignment, tighter uncertainty windows (with maintained calibration) directly reduce the set of candidate peaks per compound, improving precision. Ridge (partial pooling) produces substantially narrower intervals while keeping coverage close to nominal, and avoids the under-coverage observed with the sklearn baseline.

Why we do not prefer sklearn intervals (as-is). The sklearn ridge baseline produces very narrow intervals, but empirically under-covers the nominal 95% target. In a peak assignment pipeline that uses RT intervals as a hard filter, under-coverage becomes a direct failure mode: the true peak is excluded more often. The PyMC partial pooling model achieves a better operating point: narrow windows with coverage close to nominal, so it reduces candidate load without sacrificing recall.

Cost of hierarchy. The benefit of partial pooling comes at a large training cost: $\sim$7.8 hours (lib208) and $\sim$14.3 hours (lib209) in the original baseline run. With the chem-linear form and cached fixed Cholesky factors, the same ADVI configuration took $\sim$2.3 hours (lib208) and $\sim$4.0 hours (lib209) on this machine. If we retrain infrequently (e.g., periodic offline retrains), this may be acceptable; if we need rapid iteration, Ridge (supercategory, sklearn) remains valuable.

Cold-start behaviour. Both ridge forms can provide predictions for new species as long as the row can be assigned to an existing `species_cluster`. The hierarchical model additionally defines a principled prior over unseen `species` within a supercategory, and Section 4.5 shows it can score unseen-species rows reliably via an explicit backoff derived from fitted training coefficients. For truly unseen chemicals, Section 4.6 evaluates single-model cold-start via the chem-linear compound prior; the next improvements focus on stabilizing the chemistry head (class anchoring, stronger shrinkage, and possibly mild non-linearity) while keeping the residual δ_k so seen-compound performance is maintained. In practice, cold-start deployments may still benefit from collecting a small number of anchor labels per new matrix to avoid extreme extrapolation.

Lasso baselines. Lasso (supercategory) is generally weaker in RMSE than ridge candidates. Its interval is a fixed window, so coverage should be interpreted as "fraction inside the window" rather than probabilistic calibration.

Why we exclude the Lasso (species) baseline. The legacy "local models" lasso scores only a subset of rows and therefore requires a fallback model anyway. On realtest it scores 51.2% of lib208 rows (RMSE 0.011425) and 82.7% of lib209 rows (RMSE 0.007741), so we do not include it as a primary candidate in the selection tables.

6 Conclusion and recommendation

Based on current cap100 → realtest results, **Ridge (partial pooling, chem-linear)** is the best overall model form for peak assignment: it improves RMSE and substantially tightens predictive windows with near-nominal coverage. The main downside is training time (hours).

We recommend:

- **Primary**: Ridge (partial pooling, chem-linear) as the default going forward (seen performance maintained, and enables single-model compound cold-start via chemistry).
- **Fallback / fast iteration**: Ridge (supercategory, sklearn) as a fast, stable baseline and safety net. Avoid using its intervals for gating unless we explicitly calibrate them to hit nominal coverage.

7 Future extensions

- **Peak-assignment integration:** use the predictive distribution as a likelihood term (not only a hard gate) and tune operating points explicitly in terms of candidate count vs miss rate.
- **Calibration as a first-class step:** fit a small global (or per-supercategory) predictive-scale calibration on a held-out validation set to hit nominal coverage while minimizing window width.
- **Active learning loop closure:** choose which species/compounds to curate next based on assignment ambiguity (high candidate count), long-tail support, and calibration failures; retrain periodically and measure end-to-end impact.
- **Instrument transfer:** add an explicit transfer layer (e.g., per-instrument offset/scale, or hierarchical run-drift components) so new instruments can adapt using limited anchor data without losing calibration.
- **Cold-start matrices and unseen chemicals:** use chemistry-informed priors for compound effects (e.g., the chem-linear single-model prior in this report) and test richer embedding maps (more PCA dimensions, non-linear heads) to further improve true zero-shot accuracy while maintaining calibration.
- **Training speed and operational simplicity:** reduce ADVI wall time (structured VI, fewer parameters, better initialization) and provide clean production scripts with sensible hardcoded defaults for reproducibility.